

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

Факультет **Прикладной информатики**

Образовательная программа **Мобильные и сетевые технологии**

Направление подготовки(специальность) **09.03.03 Прикладная информатика**

ЛАБОРАТОРНАЯ РАБОТА №6

По дисциплине «Проектирование и реализация баз данных»

Тема: Работа с БД в СУБД MongoDB

Выполнила Мкртчян К.Г.; К3241.

Проверили Говорова М.М., Белов А.О.

Дата 23.05.2026

Цель работы

Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

1. Создание БД и вставка данных (Задание 2.1.1)

1. Переключаемся/создаем БД learn

```
use learn
```

2. Добавляем всех единорогов (копируйте весь блок целиком)

```
db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm',  
vampires: 63});
```

```
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires:  
43});
```

```
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm',  
vampires: 182});
```

```
db.unicorns.insert({name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires:  
99});
```

```
db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f',  
vampires: 80});
```

```
db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f',  
vampires: 40});
```

```
db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm',  
vampires: 39});
```

```
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm',  
vampires: 2});
```

```
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f',  
vampires: 33});
```

```
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm',  
vampires: 54});
```

```
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

```
db.unicorns.insert({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm',  
vampires: 165});
```

3. Проверяем, что все добавилось

```
db.unicorns.find()
```

2. Выборка и фильтрация (Задания 2.2.1 – 2.3.4)

2.2.1 Самцы и самки

```
db.unicorns.find({gender: 'm'}).sort({name: 1})
```

```
> db.unicorns.find({gender: 'm'}).sort({name: 1})
{
  _id: ObjectId('6a10acda39d76de890721d31'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a10acda39d76de890721d26'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
```

```
db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
```

```

db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
{
  _id: ObjectId('6a10acda39d76de890721d27'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
{
  _id: ObjectId('6a10acda39d76de890721d2b'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 733,
  gender: 'f',
  vampires: 40
}
{
  _id: ObjectId('6a10acda39d76de890721d2e'),

```

2.2.1 (пункт 2) Самки, любящие carrot

db.unicorns.findOne({gender: 'f', loves: 'carrot'})

```

> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
< {
  _id: ObjectId('6a10acda39d76de890721d27'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}

```

db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)

```
> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
< {
  _id: ObjectId('6a10acda39d76de890721d27'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
```

2.2.2 Самцы без поля loves и _id

db.unicorns.find({gender: 'm'}, {_id: 0, loves: 0})

```
> db.unicorns.find({gender: 'm'}, {_id: 0, loves: 0})
< {
  name: 'Horny',
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
  name: 'Unicrom',
  weight: 984,
  gender: 'm',
  vampires: 182
}
{
  name: 'Roooooodles',
  weight: 575,
  gender: 'm',
  vampires: 99
}
{
```

2.2.3 В обратном порядке добавления

db.unicorns.find().sort({\$natural: -1})

```

> db.unicorns.find().sort({$natural: -1})
< {
  _id: ObjectId('6a10acda39d76de890721d31'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a10acda39d76de890721d30'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
{

```

2.1.4 Первое любимое предпочтение

db.unicorns.find({}, {_id: 0, loves: {\$slice: 1}})

```

> db.unicorns.find({}, {_id: 0, loves: {$slice: 1}})
< {
  name: 'Horny',
  loves: [
    'carrot'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
{
  name: 'Aurora',
  loves: [
    'carrot'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}

```

2.3.1 Самки весом 500-700 кг

```
db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
```

```
> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
< {
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80
}
{
  name: 'Leia',
  loves: [
    'apple',
    'watermelon'
  ],
  weight: 601,
  gender: 'f',
  vampires: 33
}
```

2.3.2 Самцы вес >500 И любящие grape И lemon

```
db.unicorns.find({gender: 'm', weight: {$gt: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
```

```
> db.unicorns.find({gender: 'm', weight: {$gt: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
< {
  name: 'Kenny',
  loves: [
    'grape',
    'lemon'
  ],
  weight: 690,
  gender: 'm',
  vampires: 39
}
learn>
```

2.3.3 У кого нет поля vampires

```
db.unicorns.find({vampires: {$exists: false}})
```



```

> db.unicorns.find({vampires: {$exists: false}})
< {
  _id: ObjectId('6a10acda39d76de890721d30'),
  name: 'Nimue',
  loves: [
    'grape',
    'carrot'
  ],
  weight: 540,
  gender: 'f'
}
learn>

```

2.3.4 Имена самцов + первое предпочтение

`db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}}).sort({name: 1})`

```

> db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}}).sort({name: 1})
< {
  _id: ObjectId('6a10acda39d76de890721d31'),
  name: 'Dunx',
  loves: [
    'grape'
  ]
}
{
  _id: ObjectId('6a10acda39d76de890721d26'),
  name: 'Horny',
  loves: [
    'carrot'
  ]
}
{
  _id: ObjectId('6a10acda39d76de890721d2c'),
  name: 'Kenny',
  loves: [
    'grape'
  ]
}

```

3. Вложенные объекты, курсоры, агрегация (Задания 3.1.1 – 3.2.3)

3.1.1 Создаем коллекцию towns

```
db.towns.insert({name: "Punxsutawney", population: 6200, last_sensus:
ISODate("2008-01-31"), famous_for: ["phil the groundhog"], mayor: { name: "Jim Wehrle"
}});
```

```
db.towns.insert({name: "New York", population: 22200000, last_sensus:
ISODate("2009-07-31"), famous_for: ["statue of liberty", "food"], mayor: { name: "Michael
Bloomberg", party: "I" }});
```

```
db.towns.insert({name: "Portland", population: 528000, last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"], mayor: { name: "Sam Adams", party: "D" }});
```

3.1.1 (пункт 2) Мэры с партией "I"

```
db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
```

```
> db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
< {
  name: 'New York',
  mayor: {
    name: 'Michael Bloomberg',
    party: 'I'
  }
}
```

3.1.1 (пункт 3) Беспартийные мэры (party отсутствует)

```
db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
```

```
> db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
< {
  name: 'Punxsutawney',
  mayor: {
    name: 'Jim Wehrle'
  }
}
```

3.1.2 Функция + курсор для самцов

```
function maleUnicorns() { return db.unicorns.find({gender: 'm'}) }
```

```
let cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2)
```

```
cursor.forEach(function(doc) { print(doc.name) })
```

```
> function maleUnicorns() { return db.unicorns.find({gender: 'm'}) }  
< [Function: maleUnicorns]  
> let cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2)  
> cursor.forEach(function(doc) { print(doc.name) })  
< Dunx  
< Horny
```

3.2.1 Кол-во самок весом 500-600 кг

```
db.unicorns.countDocuments({gender: 'f', weight: {$gte: 500, $lte: 600}})
```

```
> db.unicorns.countDocuments({gender: 'f', weight: {$gte: 500, $lte: 600}})  
< 2
```

3.2.2 Список уникальных предпочтений

```
db.unicorns.distinct("loves")
```

```
> db.unicorns.distinct("loves")  
< [  
  'apple',      'carrot',  
  'chocolate', 'energon',  
  'grape',      'lemon',  
  'papaya',     'redbull',  
  'strawberry', 'sugar',  
  'watermelon'  
]
```

3.2.3 Количество особей по полу (агрегация)

```
db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}}]])
```

```
> db.unicorns.aggregate([{$group: {_id: "$gender", count: {$sum: 1}}}]])  
< {  
  _id: 'm',  
  count: 7  
}  
{  
  _id: 'f',  
  count: 5  
}
```

4. Обновление и удаление данных (Задания 3.3.1 – 3.4.1)

3.3.1 Сохраняем Barny

`db.unicorns.insertOne({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})`

```
> db.unicorns.insertOne({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})
< {
  acknowledged: true,
  insertedId: ObjectId('6a11d7ee4ade321c726b892e')
}
```

`db.unicorns.find({name: 'Barny'})`

```
> db.unicorns.find({name: 'Barny'})
< {
  _id: ObjectId('6a11d7ee4ade321c726b892e'),
  name: 'Barny',
  loves: [
    'grape'
  ],
  weight: 340,
  gender: 'm'
}
```

3.3.2 Обновляем Айна

`db.unicorns.update({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})`

`db.unicorns.find({name: 'Ayna'})`

```

> db.unicorns.update({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})
< DeprecationWarning: Collection.update() is deprecated. Use updateOne, updateMany, or bulkWrite.
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.unicorns.find({name: 'Ayna'})
< {
  _id: ObjectId('6a10acda39d76de890721d2b'),
  name: 'Ayna',
  loves: [
    'strawberry',
    'lemon'
  ],
  weight: 800,
  gender: 'f',
  vampires: 51
}

```

3.3.3 Raleigh любит редбул

```
db.unicorns.update({name: 'Raleigh'}, {$push: {loves: "redbull"}})
```

```
db.unicorns.find({name: 'Raleigh'})
```

```

> db.unicorns.update({name: 'Raleigh'}, {$push: {loves: "redbull"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.unicorns.find({name: 'Raleigh'})
< {
  _id: ObjectId('6a10acda39d76de890721d2d'),
  name: 'Raleigh',
  loves: [
    'apple',
    'sugar',
    'redbull'
  ],
  weight: 421,
  gender: 'm',
  vampires: 2
}

```

3.3.4 Всем самцам +5 к вампирам

```
db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}})
```

```
db.unicorns.find({gender: 'm'})
```

```
> db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
> db.unicorns.find({gender: 'm'})
< {
  _id: ObjectId('6a10acda39d76de890721d26'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 68
}
```

3.3.5 Мэр Портланда беспартийный (удаляем поле party)

```
db.towns.updateOne({name: "Portland"}, {$unset: {"mayor.party": ""}})
```

```
db.towns.find({name: "Portland"})
```

```

> db.towns.updateOne({name: "Portland"}, {$unset: {"mayor.party": ""}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.towns.find({name: "Portland"})
< {
  _id: ObjectId('6a10c1f639d76de890721d35'),
  name: 'Portland',
  population: 528000,
  last_sensus: 2009-07-20T00:00:00.000Z,
  famous_for: [
    'beer',
    'food'
  ],
  mayor: {
    name: 'Sam Adams'
  }
}

```

3.3.6 Pilot любит шоколад

```
db.unicorns.update({name: 'Pilot'}, {$push: {loves: "chocolate"}})
```

```
db.unicorns.find({name: 'Pilot'})
```

```

> db.unicorns.update({name: 'Pilot'}, {$push: {loves: "chocolate"}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.unicorns.find({name: 'Pilot'})
< {
  _id: ObjectId('6a10acda39d76de890721d2f'),
  name: 'Pilot',
  loves: [
    'apple',
    'watermelon',
    'chocolate'
  ],
  weight: 650,
  gender: 'm',
  vampires: 59
}

```

3.3.7 Aurora любит сахар и лимоны (addToSet, чтобы не было дублей)

```
db.unicorns.update({name: 'Aurora'}, {$addToSet: {loves: {$each: ["sugar", "lemon"]}}})
```

```
db.unicorns.find({name: 'Aurora'})
```



```

> db.unicorns.update({name: 'Aurora'}, {$addToSet: {loves: {$each: ["sugar", "lemon"]}}})
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
> db.unicorns.find({name: 'Aurora'})
< {
  _id: ObjectId('6a10acda39d76de890721d27'),
  name: 'Aurora',
  loves: [
    'carrot',
    'grape',
    'sugar',
    'lemon'
  ],
  weight: 450,
  gender: 'f',
  vampires: 43
}

```

3.4.1 Удаляем города с беспартийными мэрами (из Towns)

```
db.towns.remove({"mayor.party": "I"})
```

```
db.towns.find({"mayor.party": "I"})
```

```

> db.towns.remove({"mayor.party": "I"})
< DeprecationWarning: Collection.remove() is deprecated. Use deleteOne, deleteMany, findOneAndDelete, or bulkWrite.
< {
  acknowledged: true,
  deletedCount: 1
}

```

```

> db.towns.find({"mayor.party": "I"})
<

```

3.4.1 (пункт 6) Очищаем коллекцию towns

```
db.towns.drop()
```

```

> db.towns.drop()
< true

```

localhost:27017 > learn > towns

Open MongoDB shell

Documents 0 Aggregations Schema Indexes 0 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE 25 0 - 0 of 0

This collection has no data

It only takes a few seconds to import data from a JSON or CSV file.

Import data

5. Ссылки и индексы (Задания 4.1.1 – 4.4.1)

4.1.1 Создаем коллекцию зон обитания

```
db.habitats.insertOne({_id: "forest", name: "Дремучий лес", description: "Много деревьев и теней"});
```

```
db.habitats.insertOne({_id: "plains", name: "Бескрайние равнины", description: "Поля диких цветов"});
```

4.1.1 Добавляем ссылки единорогам

```
db.unicorns.update({name: "Horny"}, {$set: {habitat: {$ref: "habitats", $id: "forest"}}})
```

```
db.unicorns.update({name: "Solnara"}, {$set: {habitat: {$ref: "habitats", $id: "plains"}}})
```

```
> db.unicorns.find({name: "Horny"})
< {
  _id: ObjectId('6a10acda39d76de890721d26'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 68,
  habitat: DBRef('habitats', 'forest')
}
> db.unicorns.find({name: "Solnara"})
< {
  _id: ObjectId('6a10acda39d76de890721d2a'),
  name: 'Solnara',
  loves: [
    'apple',
    'carrot',
    'chocolate'
  ],
  weight: 550,
  gender: 'f',
  vampires: 80,
  habitat: DBRef('habitats', 'plains')
}
```

4.2. Создаем уникальный индекс

```
db.unicorns.createIndex({name: 1}, {unique: true})
```

4.3.1 Смотрим индексы

```
db.unicorns.getIndexes()
```

```
> db.unicorns.getIndexes()
< [
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

4.3.1 Удаляем все индексы, кроме _id

```
db.unicorns.dropIndex("name_1")
```

```
> db.unicorns.dropIndex("name_1")
< { nIndexesWas: 2, ok: 1 }
> db.unicorns.getIndexes()
< [ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

4.4.1 Создаем БД numbers и тестируем скорость

```
use numbers
```

```
for(i = 0; i < 100000; i++){ db.numbers.insert({value: i}) }
```

```
db.numbers.find().skip(99996).limit(4).explain("executionStats") // Без индекса
```

```
db.numbers.createIndex({value: 1}) // Создаем индекс
```

```
db.numbers.find().skip(99996).limit(4).explain("executionStats") // С индексом (скорость должна быть выше)
```

В нашем случае 100 тысяч строк не так много, поэтому вариант с индексом был медленнее на 32 мс чем без него, так как индексы так же нужно читать.

Вывод

В ходе работы были изучены основные операции с документами в MongoDB (вставка, выборка, обновление, удаление), работа с вложенными структурами и агрегация данных. Созданные индексы на коллекции numbers не показали значительного выигрыша в скорости на 100 000 документах из-за малого объема данных